UNMdP
Facultad de Ingeniería

Ing. en Informática
Programación B

Primer parcial – 8 de mayo de 2026

P.P.

Nombre y Apellido: VALENTINA SGORLO **Calificación:** Aprobado

IMPORTANTE: NO SE CORREGIRA LO QUE NO SEA LEGIBLE: PUEDE ENTREGAR EN LAPIZ SIEMPRE QUE SEA OSCURO
Serán considerados al calificar este examen la eficiencia y legibilidad de las soluciones y el uso de las características del lenguaje C y de la programación estructurada. Para facilitar el seguimiento del código, ***se sugiere numerar las llaves de cada bloque, o marcar con una línea las llaves asociadas****.*
Para aprobar es necesario obtener ***al menos 5p****, de los cuales* ***al menos 4,25p*** *deben obtenerse en el* ***inciso i)***
Para promocionar, es necesario obtener ***al menos 7p****, de los cuales* ***al menos 6p*** *deben obtenerse en el* ***inciso i)***

i) Inciso a) (4 p)	*i) Inciso b) (2,5 p)*	*i) Inciso c) (2p)*	*ii) y iii) (1,5p)*	*Calificación*
3,25	2,50	1,60	1,25	8,60

Una universidad pretende gestionar las asistencias a clases de sus alumnos. Para esto utiliza un **vector VAS** de N elementos, donde cada elemento representa una **asignatura** y es un registro compuesto por una **lista simplemente enlazada L** y un entero positivo (cantidad de inscriptos a la asignatura). Cada nodo de **L** representa una **clase**, y tiene los siguientes campos: *fecha y hora* (cadena de 14, formato aa-mm-dd hh:mm, ordenada por este campo), *cantidad de presentes*, ***sublista*** de alumnos presentes con: *nro. de matrícula (entero positivo)*.

Una ***cola C*** *almacena los registros de asistencia de uno o más días donde cada elemento contiene: código de asignatura (es el índice de la misma en el vector), fecha y hora del inicio de la clase, nro. de matrícula del alumno. Una asignatura puede dictar más de una clase en el mismo día, pero no en el mismo horario.*

Una ***lista doblemente enlazada LD*** *almacena* ***estadísticas de asistencia por asignatura****. Cada nodo contiene: código de asignatura* (ordenada por este campo), *cantidad de clases registradas, media de porcentaje de asistencia a clases.*

Se pide desarrollar un programa en lenguaje C, modularizado y utilizando TDA cuando corresponda que:

i) resuelva:

a) Procesar los datos de **C** para actualizar los datos de **VAS** (todas las asignaturas ya están en el vector) según corresponda. La clase puede no existir en **L**, en tal caso debe insertarse. Los alumnos deben insertarse al final de la sublista correspondiente, excepto cuando ya existiera en la misma; en tal caso mantenerlo en **C** conservando su orden.

b) Se cargó por error la asistencia de la clase dictada en el día y horario **DH** de la asignatura **K** en la lista **L** pero de la asignatura **X**. Corregir este error en **VAS**. (X, K, DH son datos)

c) Generar **LD** (inicialmente vacía) a partir de la información contenida en **VAS**.

ii) incluya el *main()* completo que realice las invocaciones a los subprogramas definidos en **i)** además de las declaraciones de tipos e inicialización de estructuras necesarias, e invocación a funciones de carga (que no debe desarrollar).

iii) defina el tipo de ***C*** (implementación estática) utilizado en el programa y sus elementos **PoneC()** e **IniciaC()**. Indicar en qué archivo/s iría cada definición/desarrollo.

VALENTINA SGORLON

MAIN.C

```
#INCLUDE <STRING.H>
#INCLUDE <STDIO.H>
#INCLUDE <STDLIB.H>
#INCLUDE "COLA.H"
#DEFINE MAXVEC 300

TYPEDEF STRUCT NODITO{
        UNSIGNED INT NROMAT;
        STRUCT NODITO *SIG;
}NODITO;
TYPEDEF NODITO * TCOLUMNA;
TYPEDEF STRUCT NODO{
        CHAR FECHAHORA[MAXFECHA];
        UNSIGNED INT CANTPRE;
        SUBLISTA SUB;
        STRUCT NODO *SIG;
}NODO;
TYPEDEF NODO * TLISTA;
TYPEDEF STRUCT NODOD{
        UNSIGNED INT CODASIGN, CANTCR;
        FLOAT MEDIAPORCENTAJE;
        STRUCT NODOD *ANT, *SIG;
}NODOD;
TYPEDEF NODOD * PNODOD;
TYPEDEF STRUCT{
        PNODOD PRI, ULT;
}TLISTAD;
TYPEDEF STRUCT{
        UNSIGNED INT CANTINSCRIP;
        TLISTAD ASIGN;
        TLISTAD REDASIGN;
```

```
VOID ACTUALIZOVEC ( REGASIGN V[], TCOLA *C);

VOID ARREGLACLASE ( INT CODASIGNERROR, INT CODASIGNARREGLA, CHAR FECHAHORA[],
                    REGASIGN V[]);

VOID INSERTA (TLISTA CLASEERROR, TLISTA *L);

VOID GENERALO( TLISTAD *LD, INTN, REGASIGN V[]);

VOID CARGACOLA ( *C);
VOID CARGAVEC ( REGASIGN V[], INT *N);
```

```
INT MAIN(){
    INT N, CODASIGNERROR, CODASIGNARREGLA;
    TCOLA C;
    REGASIGN V[MAXVEC];
    (illegible) CHAR FECHAHORA[MAXFECHA];
    TLISTAD LD;
    CARGACOLA(&C);
    CARGAVEC (V, &N);
(a) ACTUALIZOVEC (V, &C);
    (illegible)
    PRINTF("Ingrese codigo de clase a corregir, codigo de clase
            a insertar y la fecha-hora [M]", &CODASIGNERROR, &CODASIGNARREGLA,
            FECHAHORA); Debe leer en un SCANF
(b) ARREGLACLASE( CODASIGNERROR, CODASIGNARREGLA, FECHAHORA, V);
(c) GENERALO( &LD, N, V);
    RETURN 0;
}
```

a)

```
void ActualizaVec (RegAsign V[], Tlista *C) {1}
                                  ?? → Debe declararlo y aparte inicializar el campo.
TElemLista Asist, Centinela.codigoAsign = -1;
           ActClase, AntClase, NuevaClase;
Sublista ActAlumno, AntAlumno, NuevoAlumno.

PonC (C, Centinela);
SacaC (C, &Asist);
while (Asist.codigoAsign != Centinela.codigoAsign) {2}
    ACTCLASE = V[Asist.codAsign].Asign;

    while (ActClase != NULL && strcmp(Asist.FechaHora, ActClase.FechaHora) > 0) {
        AntClase = ActClase;
        ActClase = ActClase->sig;
    }

    if (!(ACTCLASE != NULL && strcmp(Asist.FechaHora, ActClase.FechaHora) == 0)) {
        NuevaClase = (Tlista) malloc (sizeof(Nodo));
        strcpy (NuevaClase->fechahora, Asist.FechaHora);
        NuevaClase->cantPre = 0;
        NuevaClase->sub = NULL;

        if (V[Asist.codAsign].Asign == NULL ||
            strcmp (V[Aux.codAsign].Asign->fechaHora, Asist.FechaHora) > 0) {
            NuevaClase->sig = V[Aux.codAsign].Asign;
            V[Asist.codAsign].Asign = NuevaClase;
        } else {
            AntClase->sig = NuevaClase;
            NuevaClase->sig = ActClase;
        }
        ActClase = NuevaClase;
    }
    ActAlumno = ActClase->sub;
```

Modularizar algunas búsqueda o inserción

Dado que ya busco, basta con preguntar si ActClase == V[Asist.codAsig].Asign; para insertar al principio

En ambos casos, nuevaClase->Sig = actClase es correcto y podría extraerse del if.

```
while ( ActAlumno != NULL && Asist.NroMat != ActAlumno->N
        AntAlumno = ActAlumno;
        ActAlumno = ActAlumno->sig;
}
if ( ActAlumno == NULL){
        NuevoAlumno = (Sublista) MALLOC ( SIZEOF(NODO1TO));
        NuevoAlumno->NROMAT = ASIST.NROMAT; NuevoAlumno->sig = NULL;
        IF ( ActClase->sal == NULL)
                ActClase->sal = NuevoAlumno;
        ELSE
                AntAlumno->sig = NuevoAlumno;
        ActClase->CANTPRE++;
}
Si lo encuentra Ponerlo en la Cola!
(Para qué el centinela sino?)
SACA(C, &ASIST);
```

(1) (2)

VALENTINA SCORZO

3) /5

¿f 'cambia el Nombre dado en la consigna?

b) VOID ARREGLACLASE(INT CODASIGNERROR (x), INT CODASIGNARREGLA (k), CHAR FECHAHORA[] (DH), RegArgm V[]){

TLista ActClaseError, AntClaseError, ClaseError;

ActClaseError = V[CODASIGNERROR].ASIGN;
while(STRCMP(ActClaseError → FECHAHORA, FECHAHORA) < 0){
    ANTCLASEERROR = ACTCLASEERROR;
    ACTCLASEERROR = ACTCLASEERROR → sig;
}

Innecesario
CLASEERROR = ACTCLASEERROR;
IF(ACTCLASEERROR == V[CODASIGERROR].ASIGN){
    V[CODASIGNERROR].ASIGN = V[CODASIGNERROR].ASIGN → sig;
ELSE
    ANTCLASEERROR → sig = ACTCLASEERROR → sig;

INSERTA(CLASEERROR, &V[CODASIGNARREGLA].ASIGN);
}

VOID INSERTA(TLista CLASEERROR, TLista *L){
TLista Act, Ant;
Act = *L;
while(Act != NULL && STRCMP(CLASEERROR → FECHAHORA, ACT → FECHAHORA) > 0){
    ANT = ACT; ACT = ACT → sig;
}
IF(ACT == *L){
    CLASEERROR → sig = *L;
    *L = CLASEERROR;
ELSE{
    ANT → sig = CLASEERROR;
    CLASEERROR → sig = ACT;
}

→ Extraer del if

}

c)

```
VOID GENERALD ( TLISTAD *LD, int N, RegAsign V[]){
 INT i, CANTCLASES;
 PNODOD NUEVAASIGN;
 TLISTA ACTCLASE;
 FLOAT ACUMPORCENTAJE;
 (*LD).pri = (*LD).ult = NULL;

 FOR( i=0; i<N; ++i){
     ACTCLASE = V[i].ASIGN;  CANTCLASES=0; ACUMPORCENTAJE=0;
     WHILE( ACTCLASE != NULL){
         ACUMPORCENTAJE += ACTCLASE->CANTPRE / V[i].CANTINSCRIP *100;
         CANTCLASES++;
         ACTCLASE = ACTCLASE->SIG;
     }
     NUEVAASIGN = (PNODOD) MALOC (SIZEOF(NODOD));
     NUEVAASIGN->CODASIGN = i;
     NUEVAASIGN->CANTCR = CANTCLASES;
     IF( ! CANTCLASES)
         NUEVAASIGN->MEDIAPORCENTAJE = ACUMPORCENTAJE/CANTCLASES;
     ELSE
         NUEVAASIGN->MEDIAPORCENTAJE = 0;

     IF( (*LD).pri == NULL){
         NUEVAASIGN->ANT = NULL;
         NUEVAASIGN->SIG = NULL;
         (*LD).pri = (*LD).ult = NUEVAASIGN;
     }
     ELSE{
         NUEVAASIGN->SIG = NULL;
         NUEVAASIGN->ANT = (*LD).ult;
         (*LD).ult->SIG = NUEVAASIGN;
         (*LD).ult = NUEVAASIGN;
     }
 }
}
```

OBS:
Si crearz el nodo antes de recorrer podriz contar directamente en nuevzAsign->CantCR

(Castear a float)

Podriz extraerlo del while (constante)

→ Repetidas!! :) !!

obs Podría hacerlo una vez la final?

(5)

"colas.h"

```
#DEFINE MAXC 50
#DEFINE MAXFECHA 15

TYPEDEF STRUCT{
    unsigned INT OCUPADO;
    CHAR FECHAHORA[MAXFECHA];
    UNSIGNED INT NUMAT;
}TELEMENTOC;

TYPEDEF STRUCT{
    TELEMENTOC DATOS[MAXC];
    INT pri, ult;
}TCOLA;
```

obs: Prototipos?

"colas.c"

```
#INCLUDE "colas.h"

VOID PONEC(TColo *C, TElemento x){
IF((*C).ult != MAXC-1){
        IF((*C).pri == -1)
            (*C).pri++;
        (*C).DATOS[++(*C).ult] = x;
}
}

VOID INICIAC(TColo *C){
(*C).pri = (*C).ult = -1;
}
```